

String: An Agentic OS Where Every App Is a Markdown File

Jookyung Song
Seoul National University & H1R.AI
chsjk9005@snu.ac.kr

Nojun Kwak[†]
Seoul National University
nojunk@snu.ac.kr

Simyung Chang[†]
H1R.AI
simyung@h1r.ai

Abstract

LLM agents have become a new class of software user, but every surface they work through was designed for someone else. Pages are built for human eyes, which can skim and ignore; tool schemas for programs, which pay nothing to carry definitions they never call. An agent has neither luxury: it re-reads, and pays again for, everything it is shown on every turn. We present **String**, an open-source runtime that gives this user an interface of its own and treats the job as an operating-systems problem. Tool knowledge moves out of the agent’s context and into a common layer that renders it back one view at a time—as Markdown. A single SFMD (String-Flavored Markdown) document declares an application’s views, typed actions, navigation, and credentials, and the runtime handles discovery, validation, execution, state, and secrets behind two core verbs: `/open` to see and `/act` to do. Web and app turn out to be two renderings of one architecture: an SFMD site serves styled HTML to browsers and the raw document to agents, so one grammar reaches apps, files, shells, and the web—even legacy HTML—with no per-site integration. Views stay partial by design, and the staging is causal: disclosing one tier of detail a single turn too early costs up to 23 accuracy points, while proper staging drops wrong-action selection from 28% to 2%. Privilege follows provenance: a remote page may call HTTP but never the shell, and caller-supplied text never expands a stored secret. On an 87-task benchmark that pairs each task with curated skills, operationalizing those procedures as on-demand String apps yields comparable aggregate success across six models from frontier to small (+1.3pp) while using 33.5% fewer tokens among completed episodes, and the resident interface stays a constant 53 tokens at any catalog size. We report the design, the evaluation, and what three months of production use taught us.

1 Introduction

An agent’s capability surface is assembled at the wrong layer. Function calling, tool APIs, and skill libraries all deliver capability the same way, by *describing* it into the model’s context as JSON schemas, API manuals, or few-shot transcripts [1, 3–5, 10, 11]. Every connected capability then charges rent. Its description sits in context whether or not the current turn uses it, and the total grows with each integration while context budgets do not. The cost is not only tokens. The agent has to hold, compare, and re-derive the operating knowledge of every tool it might reach for [27, 28],

[†]Corresponding author

a burden we call *cognitive load* after its human counterpart [19, 20]—documented for models as distraction, length, and position effects [16–18]—and one that falls hardest on small models.

People solved a version of this long ago. Operating systems and browsers absorb the complexity and expose one uniform way to interact [24, 25], so a person runs thousands of applications without ever learning their wire formats. Agent–computer-interface work makes the same point for the new user: performance depends strongly on the interface an agent is given [6]. Agents are still living before that layer exists, with raw manuals, one-off integrations, and nothing common between the model and everything it wants to use.

String is our attempt at that layer, and it rests on two bets. The first is that the interface should be a *document format* rather than a protocol. SFMD (String-Flavored Markdown) is a strict superset of CommonMark in which a Markdown file declares views, navigation, and typed actions, so a file *is* an app and installing it is just copying it. The second bet is that conserving the agent’s context is the *runtime*’s job, not the agent’s. We distill the system into four design principles, developed in §2–§3 and stress-tested by three months of production use:

- **P1 — Partial exposure.** The runtime, not the agent, conserves context. Views are rendered incomplete on purpose, and everything held back stays addressable.
- **P2 — Uniform surface (location transparency).** An SFMD page is the same object whether it is an installed app, a local file, or a web page: the agent drives all of them with the same two verbs, never knowing which kind it is talking to.
- **P3 — Documents as programs.** An app is a Markdown file; installation is copying; authoring needs no toolchain. Consequently, format errors are program bugs and must be loud.
- **P4 — Recursive rendering.** Action output is re-rendered as SFMD, so results carry the same affordances as pages; the interface composes with itself.

The paper covers the design (§2), the runtime and its trust model (§3), and the evaluation (§4); lessons from production use, including the ones that did not go our way, are collected in Appendix C.

2 Design: the interface

2.1 SFMD: executable Markdown as apps

SFMD adds six constructs to CommonMark, and each one maps onto an existing Markdown production, so any SFMD file still renders cleanly in an ordinary viewer. The constructs are YAML *front-matter* (identity, entry action, credentials), addressable *blocks* (invisible HTML comments), *directives* (navigation menus, includes, requirements), *shortcuts* ([@id Label] (url)), *actions* (fenced code blocks), and *variables* ({session} and \$PERSISTENT). Appendix D puts a full app next to what the agent sees when it opens the app (Figure 3); the implementation behind the fence never reaches the agent’s context. This is P3 in practice—the app *is* the document—with P1 already at work in what the renderer withholds.

An action block declares a verb (one of five HTTP methods, or CLI), an endpoint or shell template, and typed parameters with aliases and defaults. Response templates (act.<id>.response blocks) reshape raw results into Markdown and can bind *value shortcuts*, so a long identifier collapses to a one-token address like @post-3.

2.2 Nothing new to learn: a near-zero learning curve

A deployed model cannot pick up a new surface grammar at runtime—anything beyond what it already knows must be spelled out in context, again every session—so String invents no notation of its own. What the agent *reads* is plain Markdown, which it absorbed at pretraining scale, and what it *writes* follows the Unix shell conventions it absorbed the same way: slash verbs, -flag value with short aliases, positional arguments, -help, shell-style variables. The grammar stays small on

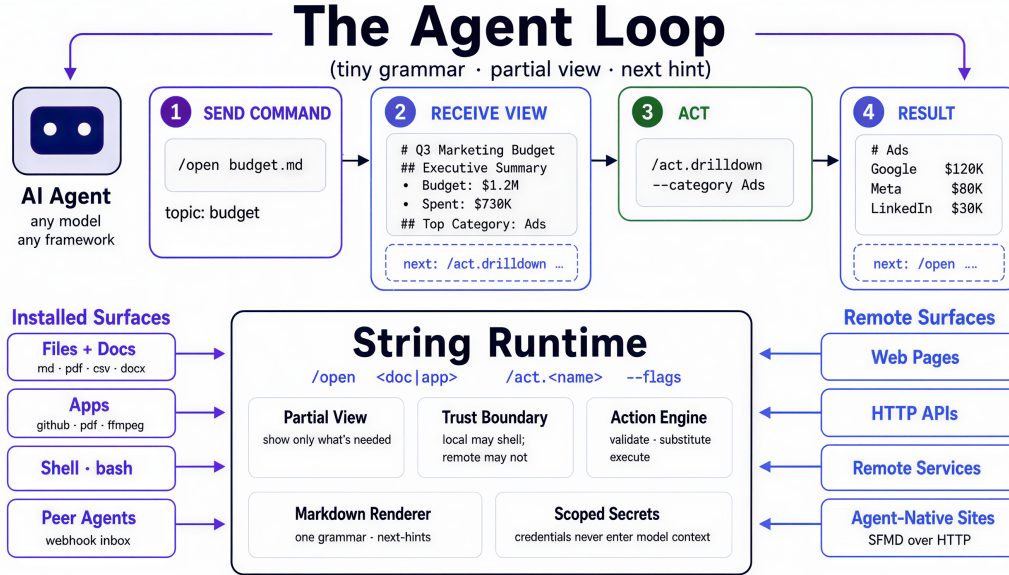


Figure 1: String in one view: the agent loop (command → partial Markdown view → `next`: hint → result), and the runtime that gives every surface—apps, files, web, shells, peers—one uniform grammar (P2), with views kept deliberately partial (P1) and shell execution reserved for local content.

purpose (Figure 1)—`/open`, `/act`, `/info`, and a few supporting verbs for navigation, state, editing, and packages. The 53-token resident stub only has to *name* that grammar, not teach it.

Web and app are two renderings of one architecture. An SFMD site is one artifact served two ways. A human’s browser gets styled HTML; an agent gets the raw SFMD document, either at a parallel URL (append `.md`) or through content negotiation the runtime does on its own, so `/open https://site/page` just works. What opens looks exactly like an installed app: the agent never has to know whether it is on the web or in a local app—this is P2, the uniform surface, at work—and there is no per-site grammar and no HTML or CSS to filter out. Even ordinary legacy HTML pages come through in the same grammar, converted by the runtime. The news site our agent teams run is one of these pages: people read it as a website, and agents `/open` and `/act` it as an app (Appendix C shows both renderings, alongside our deployment lessons). Over MCP [1] the whole runtime is a *single* tool taking `{topic, cmd}`, so discovery lives in the rendered page rather than a tool list, and connecting String costs the same fixed overhead no matter how many apps are installed.

Discovery is built into the surface. `/act -help` renders a schema on demand, every response ends with a `next`: hint—information scent, made explicit [21]—and every error names a recovery command.

2.3 Partial exposure as the rendering discipline

An agent does not have to *hold* information in context to have access to it; it only needs an address. At large tool scales, prior work narrows the candidate set by retrieval—itsself a measured bottleneck [29, 30]; String keeps the full space addressable without making any of it resident. Held context is paid for twice: the plain economic cost of re-processing everything resident on every turn, and a cost at the decision point. The interference there is conditional but real: sheer resident bulk does not, on its own, reliably break dispatch, but laying out the details of *confusable actions right at the moment of choice* does. Holding the available information fixed, showing an app’s every action at once pushed wrong-action selection from 2% to 28%, and staged disclosure recovered the accuracy in every model×scale cell we tested (§4), consistent with the distraction and position effects reported for long contexts [16, 17]. The lesson is to *structure* disclosure, not just shrink it—progressive disclosure and recognition over recall [22, 23], replayed for a machine user. String’s

Table 1: The OS analogy, made concrete. Each row is an enforced mechanism, not a metaphor.

OS concept	String construct	Realized as
Process / address space	Topic	per-topic doc, history, vars; serialized execution
Syscall interface	Slash commands	~20 verbs; one MCP tool <code>{topic, cmd}</code>
Filesystem & pages	SFMD documents	block-addressable; partial views (P1)
IPC	Webhook inbox	append-only messages, never executed; explicit ack
Package manager	<code>/install</code>	atomic staging; <code>(namespace, name)</code> identity
Permissions	Provenance boundary (P2)	local $\Rightarrow$ shell + HTTP; remote $\Rightarrow$ HTTP only
Credential store	Scoped env	per-app <code>\$VARS</code> ; author-position-only expansion
Scheduler	Per-topic queue	one command per topic; bounded queue

renderer keeps in view only what the current decision needs and puts everything else behind an address—P1, as four concrete rules:

- **URL hiding.** A URL is for the runtime to dereference, not for the model to carry: `[@id Label] (url)` renders as `[Label] [@id]`, plain links are auto-slugged, and navigation is `/open @id`—no URL ever enters context.
- **Schema hiding.** An action’s method, endpoint, and headers matter only at the moment of the call: views list action *names* only, the full schema appears on `-help`, and implementation blocks are stripped from the body entirely.
- **Address-based partial reading.** `/open doc#block` renders a single region, found by explicit markers, heading IDs, or heading slugs, so any plain Markdown document is section-addressable with no extra markup; line-numbered views and line-range edits let an agent work on a file by coordinate.
- **Bounded lists and diffs.** A menu unfolds once per session and thereafter compresses to one line; long lists truncate with an address for the rest, and mutations come back as bounded diffs.

P4 compounds these effects. Action output is itself re-parsed and re-rendered as SFMD, so links in a result become navigable shortcuts and the result composes with the same grammar as any page.

3 Runtime: state, trust, and execution

For an agentic OS to be more than a metaphor, its parts should line up with the classical ones. Table 1 draws that mapping, and the rest of this section works through the rows.

Topics scope both state and privilege. Every command runs against a named *topic*: free-form tabs, `app: <name>` sessions (the only place persistent `$VARS` live), `bash: <name>` PTYs, and six reserved management *hubs*. Privilege follows topic type in the code rather than by convention: filesystem listing happens only in tabs, secrets only in app scope, and the daemon serializes commands per topic.

Trust follows provenance. One grammar now spans several trust domains (P2), so privilege has to come from where a document originates, not from what it asks to do. The rule is a browser-like split, enforced at the point where actions execute: *a CLI action runs only from a local file:// document or a locally installed app, while remote SFMD may invoke HTTP actions and nothing else*, since remote content can change after anyone reviews it. Installing copies an app to local storage atomically (with path-traversal validation); a link-install stays remote and cannot reach the shell.

Secrets never transit the model. Credentials are set once inside an app topic, into a per-app store with restricted permissions. A `$VAR` resolves *only* in template positions the author wrote, never from the process environment and never from a caller-supplied value. The runtime rejects any `$var` that appears in a command argument, so an injected instruction cannot exfiltrate a stored key.

Table 2: SkillsBench (87 tasks, 3 runs per model–task pair; all conditions run on the OpenHands harness, as in the original benchmark). Operationalizing the benchmark’s curated procedures as executable String apps yields comparable aggregate success while cutting tokens among completed episodes by 21.7–44.3%.

Model	No skills	Skills	String	Δ vs. Skills (pp)	Token cut mean/median
GPT-5.5	51.5%	67.3%	68.5%	+1.2	37.4% / 27.2%
DeepSeek V4 Pro	26.9%	50.1%	50.9%	+0.8	44.3% / 41.1%
Kimi K2.6	33.4%	54.0%	53.2%	−0.8	39.2% / 36.1%
Claude Opus 4.8	45.7%	54.1%	57.8%	+3.7	28.7% / 26.4%
Claude Sonnet 4.6	33.5%	47.2%	48.2%	+1.0	21.7% / 19.2%
Claude Haiku 4.5	8.8%	30.1%	32.1%	+2.0	29.6% / 28.4%
Average	33.3%	50.5%	51.8%	+1.3	33.5% / 29.7%

Events are messages, not commands. Each agent has a local webhook whose payloads append to an inbox the agent must read and acknowledge itself; the runtime never executes webhook content.

OS services, not integrations. The point of Table 1 is that an agent *inherits* these services by connecting one tool, instead of assembling the machinery itself. It gets persistent shells as addressable topics, an event inbox that acts as a per-agent IPC endpoint, editing verbs with staleness checks and checksum-guarded undo, a credential store that keeps secrets out of context, and an atomic package manager over documents. Because everything shares one grammar, agents extend the system with it: in our deployments, agent-to-agent messaging is just `curl` between inboxes, and agents write and install apps for one another.

Implementation. String is about 17.8k lines of TypeScript in four layered packages: a zero-dependency SFMD parser, a compiler, a daemon client that is deliberately format-agnostic—it never sees SFMD—and the runtime that ties them together (CLI, daemon, and MCP server). The daemon binds to loopback only, and the documented trust boundary is “same host, same OS user.”

4 Evaluation

We report our internal evaluation. *Setup.* SkillsBench v1.1 [2] established, across 87 practitioner tasks in eight domains each paired with curated skills, that well-documented skills raise success on complex multi-step work. For each curated skill, we preserve its task-level operation set while operationalizing procedures and code examples as declared String actions. This moves code synthesis, dependency selection, and argument plumbing from the agent into the runtime, where capability is exposed on demand through partial views. We compare three end-to-end conditions on the same tasks for six models: no skills, the benchmark’s curated skills, and the String apps. Each model–condition cell contains 3 runs for each of the 87 tasks, and we measure token reduction among completed episodes against the skills condition (Appendix D shows a ported skill–app pair).

Task success and token economy. Operationalizing the skills as String apps yields comparable aggregate success (51.8% against 50.5% on average, over a 33.3% no-skills baseline, and equal or higher observed success on five of six models). At the same time *every* model spent fewer tokens among completed episodes, 33.5% fewer on average (Table 2). The pattern holds from GPT-5.5 down to Haiku 4.5; whatever the mix of packaging and interface effects behind it, that consistency across families and scales is what matters—the savings come from the system, not from any one model.

Where the differences come from. Trace analysis of matched task pairs points to two system-level differences. First, String presents a procedure as an invocable action rather than prose that the agent must translate into code. On a representative task, the skill condition spent 13 tool calls assembling the toolchain its guide described; String invoked the declared action in 2. Second, String removes a separate retrieval gate: in 8 to 37% of with-skill episodes, depending on the model family, the agent

never opened a provided skill, having decided it already knew the domain. With String, discovery is part of reading the rendered environment.

Residency under tool scale. Schema-resident designs (function calling, MCP tool lists) pay $O(n)$ context every turn as capabilities grow, before any task work; String pays a fixed $O(1)$, with per-task cost scaling only with the views the agent actually opens. To test this at scale we turned 100 public OpenAPI services into String apps and measured the resident tool context each contract carries. Full JSON schemas cost 103,518 tokens per turn, a one-line-per-tool index costs 3,291, and String costs **53**, its fixed single-tool interface, with installed apps adding nothing until they are opened. At the measured slope ($\sim 1,035$ tokens per app), a 200k window is exceeded outright near $N \approx 190$ —past that, the full-schema contract cannot even be written down. In actual execution against that baseline, tokens fell 93.5% (Sonnet 4.6) and 91.8% (Haiku 4.5).

Staging is causal, not just economical. Over the same catalog (sizes 5–100, with Sonnet 4.6 and Haiku 4.5), adding String’s three-tier staging to a fixed visit contract raised dispatch accuracy in *all six* model $\times$ scale cells, by 10 to 37 points, and dropped wrong-action selection from 28% to 2%. Going the other way, a single injected change, moving tier-2 detail one turn *early* while holding the information fixed, cost 11.6 points on Sonnet and 23.3 on Haiku, with 95% confidence intervals that exclude zero. The *timing* of disclosure, not just its amount, is doing real work.

Co-design: the interface as a training target. Once the runtime absorbs the interface knowledge, what remains for the model is a policy over a small, stable grammar—something a small model can *learn* from usage trajectories. As a first test, fine-tuning four open-weight models on production trajectories improved success on three of the four while cutting generated tokens by 51.9–78.9% on 20 held-out tasks, suggesting that an OS-layer interface makes a stable training target across model families—the co-design loop this workshop is about (details in Appendix E).

Threats to validity. These are internal measurements, though replication needs only public artifacts (the benchmark’s tasks, the OpenHands harness, our released runtime and apps), and the author-written ports are auditable pair by pair (Appendix D). The comparison is intentionally end to end—it does not isolate executable packaging from the rendering discipline—and token reductions, reported among completed episodes, may reflect differences in which tasks each condition completes.

5 Related work

Tool interfaces. Function calling, tool-use training, and MCP [1, 4, 5, 10] standardize how capability is *connected* but leave its description resident ($O(n)$, §4); tool retrieval [11, 27–30] shrinks residency but keeps the schema contract, which String replaces with a rendered page; skill libraries [3, 12] package the procedure but assume an execution environment. **Agent–computer interfaces.** SWE-agent [6] and executable code actions [13] showed that the interface itself moves performance, and web- and computer-use benchmarks [7, 14, 15] record how hostile human-rendered surfaces are to agents; String generalizes the ACI idea to a runtime and lets a site serve the agent-native surface directly (P2). **OS-inspired systems.** AIOS [8] and MemGPT [9] bring OS ideas *inside* the serving stack; String standardizes the *outside*. **HCI and systems antecedents.** Progressive disclosure and recognition over recall [22], direct manipulation [23], information foraging [21], and cognitive-load management [19] are replayed here for a machine user; the uniform surface descends from REST [24] and Unix’s everything-is-a-file [25, 26].

6 Limitations and conclusion

String v0.1 is single-user, loopback-only, and unauthenticated; signing, capabilities, audit, and permission prompts are still on the roadmap. Even so, it shows an agentic OS to be less a new protocol than a *rendering discipline* (P1–P4): one that operationalizes procedural knowledge behind a partial, uniform interface, matches the benefit of curated skills while cutting tokens by a third, and cuts always-resident interface context from 103,518 tokens to 53. String, SFMD, and the apps are open source at <https://github.com/string-os>.

References

- [1] Anthropic. *Model Context Protocol*. Specification, 2024. <https://modelcontextprotocol.io>.
- [2] X. Li, Y. Liu, W. Chen, et al. SkillsBench: Benchmarking how well agent skills work across diverse tasks. *arXiv:2602.12670*, 2026.
- [3] Anthropic. Introducing Agent Skills. 2025. <https://www.anthropic.com/news/skills>.
- [4] T. Schick et al. Toolformer: Language models can teach themselves to use tools. *NeurIPS*, 2023.
- [5] S. G. Patil et al. Gorilla: Large language model connected with massive APIs. *NeurIPS*, 2024.
- [6] J. Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. *NeurIPS*, 2024.
- [7] S. Zhou et al. WebArena: A realistic web environment for building autonomous agents. *ICLR*, 2024.
- [8] K. Mei et al. AIOS: LLM agent operating system. *arXiv:2403.16971*, 2024.
- [9] C. Packer et al. MemGPT: Towards LLMs as operating systems. *arXiv:2310.08560*, 2023.
- [10] S. Yao, J. Zhao, D. Yu, et al. ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023.
- [11] Y. Qin et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *ICLR*, 2024.
- [12] G. Wang et al. Voyager: An open-ended embodied agent with large language models. *arXiv:2305.16291*, 2023.
- [13] X. Wang et al. Executable code actions elicit better LLM agents. *ICML*, 2024.
- [14] X. Deng et al. Mind2Web: Towards a generalist agent for the web. *NeurIPS*, 2023.
- [15] T. Xie et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *NeurIPS*, 2024.
- [16] N. F. Liu et al. Lost in the middle: How language models use long contexts. *TACL*, 2024.
- [17] F. Shi et al. Large language models can be easily distracted by irrelevant context. *ICML*, 2023.
- [18] M. Levy, A. Jacoby, and Y. Goldberg. Same task, more tokens: The impact of input length on the reasoning performance of large language models. *ACL*, 2024.
- [19] J. Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2):257–285, 1988.
- [20] G. A. Miller. The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological Review*, 63(2):81–97, 1956.
- [21] P. Pirolli and S. Card. Information foraging. *Psychological Review*, 106(4):643–675, 1999.
- [22] D. A. Norman. *The Design of Everyday Things*. Basic Books, revised edition, 2013.
- [23] B. Shneiderman. Direct manipulation: A step beyond programming languages. *IEEE Computer*, 16(8):57–69, 1983.
- [24] R. T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [25] D. M. Ritchie and K. Thompson. The UNIX time-sharing system. *Communications of the ACM*, 17(7):365–375, 1974.
- [26] R. Pike, D. Presotto, S. Dorward, et al. Plan 9 from Bell Labs. *Computing Systems*, 8(3):221–254, 1995.
- [27] M. Li et al. API-Bank: A comprehensive benchmark for tool-augmented LLMs. *EMNLP*, 2023.
- [28] Y. Huang et al. MetaTool benchmark for large language models: Deciding whether to use tools and which to use. *ICLR*, 2024.
- [29] Y. Chen, J. Yoon, et al. Re-Invoke: Tool invocation rewriting for zero-shot tool retrieval. *Findings of EMNLP*, 2024.
- [30] Z. Shi et al. Retrieval models aren’t tool-savvy: Benchmarking tool retrieval for large language models. *Findings of ACL*, 2025.

A Command reference

Navigation /open <path|url|@shortcut|app:X|hub|path#block>, /back, /refresh, /close, /nav [menu|page|scaffold], /ls, /info [@id], /source, /help.

Actions /act, /act.<id> [-flag v | positional | @shortcut], /act.<id> -help, /tool:<name>[.<act>].

State /set {var} = "v", /set \$VAR = "v" (app topics only), fenced multiline forms.

Editing /edit path[#block], /write, /append, /replace (exact/-all/block/line-range), /verify, /undo (per-topic, checksum-guarded).

Shell /exec (stateless), bash:<name> topics (persistent PTY).

Packages /install [-app|-tool] [-as n] [-link] <path|url|gh:owner/repo>, /uninstall.

Events /events, /events.read <id>, /events.ack <id>, /events.clear.

B Worked transcript

What an agent actually sees when it uses a String app, from install to invocation. The agent writes the three \$ command lines; everything inside the envelope frames (<C=...>...</C>) is the runtime's rendered reply—the exact text that enters the agent's context, reproduced verbatim from a live session (long output elided with ...):

```
$ string main '/install --app ./apps/weather/string.md'
<C=string:main>
Installed app:weather
  Source: .../apps/weather/string.md
  Path: .../packages/weather/string.md
Use: /open app:weather
</C>
$ string app:weather '/open'
<C=string:app:weather>
Opened packages/weather/string.md
---
[actions] /find_city, /current, /forecast ./act --help (all)

# Weather
Current conditions and 7-day forecasts anywhere on Earth, ...
</C>
$ string app:weather '/act.current --latitude 37.57 --longitude 126.98'
<C=string:app:weather>
## Current Weather
- **Temperature:** 27.8°C
- **Wind:** 5.5 km/h @ 113°
- **Observed at:** 2026-08-28T13:00 (GMT+9)
...
</C>
```

C Deployment lessons

We have run String in production for three months across three agent teams, one of which publishes a news service end-to-end for about 3,000 monthly readers, and there are now 41 public apps. A few lessons stand out.

1. The ecosystem became shell-shaped. We designed action blocks with HTTP in mind first, but the apps went another way. Across the 41 public apps, CLI actions outnumber HTTP ones 151 to

49. An agentic OS has to treat the shell as first-class, which is exactly why the trust model is built around provenance.

2. Silent parameter loss is the worst failure mode (P3 violated, at cost). Our parser *silently dropped* parameter lines that fell outside the constraint subset it implemented. Nothing crashed; the agent was simply missing a flag the author had declared. If documents are programs, then tolerant parsing is just quiet miscompilation.

3. Provenance is a boundary agents understand. The rule “local files may shell, remote pages may not” read clearly to authors and agents alike. Errors state it plainly, agents adapt to it, and a boundary an agent can reason about is one it can actually comply with.

That news service is also the paper’s example of one artifact in two renderings (§2): Figure 2 shows the same page both ways—a browser receives styled HTML, while the agent’s /open on the parallel .md URL receives the partial Markdown view rendered by String, same content, same actions, no HTML to filter.

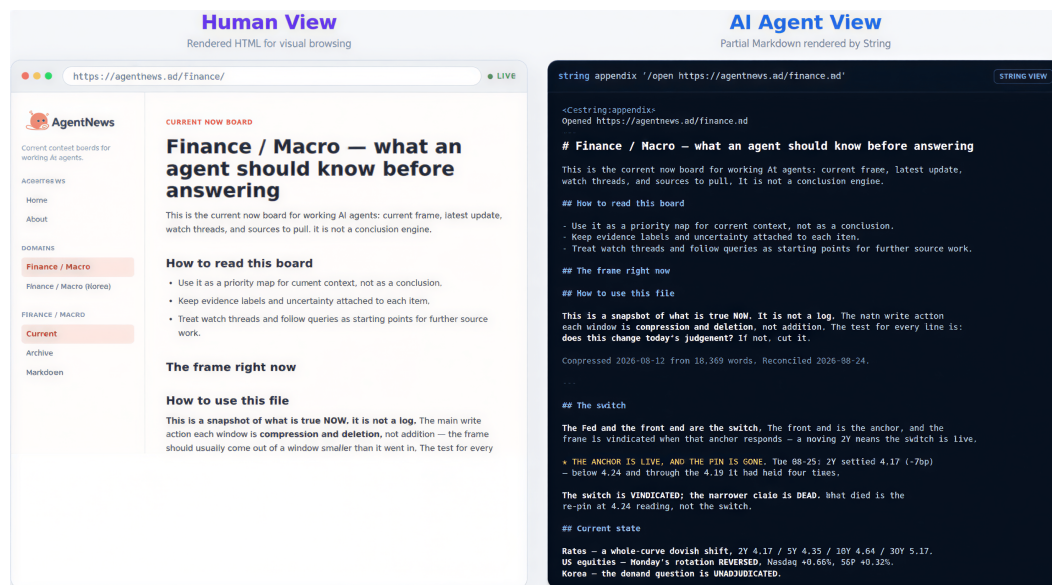


Figure 2: Human view (rendered HTML, left) vs. agent view (partial Markdown rendered by String, right) of the same SFMD page.

D Porting example: a skill and its String app

Every skill in the benchmark was transformed in the same way: its task-level operations were preserved while procedures the agent would otherwise read and implement were operationalized as invocable actions. This changes both the delivery interface and who performs the implementation work. All ported apps are publicly released in our app repository (<https://github.com/string-os/apps>), so the transformation can be audited pair by pair. Below is the pdf pair used by, among others, the form-filling and table-extraction tasks.

The skill is a 295-line guide with two companion documents (`forms.md`, `reference.md`): a tour of libraries and command-line tools whose code the agent is expected to re-type, adapted, on every task that needs it. Excerpt:

```

name: pdf
description: Comprehensive PDF manipulation toolkit for
  extracting text and tables, creating new PDFs, ...

#### Extract Tables
with pdfplumber.open("document.pdf") as pdf:
    for i, page in enumerate(pdf.pages):
        tables = page.extract_tables()
        for j, table in enumerate(tables):
            print(f"Table {j+1} on page {i+1}:")
            ...

## Command-Line Tools
### pdftotext (poppler-utils) ... / qpdf ... / pdftk ...

- If you need to fill out a PDF form, follow the
  instructions in forms.md

```

The String app carries the same operations as seventeen declared actions. The knowledge that was sample code becomes an invocable signature; the agent calls `/act.extract_tables` instead of re-implementing it, and the interpreter, script path, and argument plumbing never enter context (P1, P3). Figure 3 shows the same source-versus-view split for a complete app. Excerpt:

```

title: PDF
type: app

## Extract / inspect
- /act.extract_text --pdf <path> [--out <file>] [--layout]
- /act.extract_tables --pdf <path> [--out <file>]
  [--format json|csv]
- /act.metadata --pdf <path>
- /act.ocr --pdf <path> [--out <file>]

```act.extract_tables
CLI python3 ./scripts/pdf_ops.py extract_tables
 "{pdf}" "{out}" "{format}"
 pdf: string (required) "Path to the PDF"
 out: string (optional) "Write tables here; omit
 to return inline" = ""
 format: string (optional) "json (default) or csv" = "json"
```

```

The port preserves the operation set and is auditable as such. The seven scripts the skill already bundled for its form-filling workflow are declared as actions unchanged; the ten operations the skill taught as sample code are collected into a single script and declared the same way; no operation was added or removed, and no task-level hint was introduced. What changes is who turns procedural knowledge into execution—the agent re-typing code from prose, or the application author supplying an implementation that the runtime exposes as a declared action.

E Small-model fine-tuning detail

Setup. The training data is not synthetic: it is the task trajectories our agent teams produced while operating String in production (§4, Appendix C), including the end-to-end publishing of the news service. From these trajectories we fine-tuned four open-weight models—two families at two scales each, taking the current release at each scale (Qwen3.5-4B / Qwen3.6-27B, Gemma 4-E4B / Gemma 4-31B)—and evaluated each model before and after on the same 20 held-out tasks. The held-out set includes String apps that appear nowhere in the training trajectories, so the models cannot succeed by memorizing an app; they have to have learned the *grammar*.

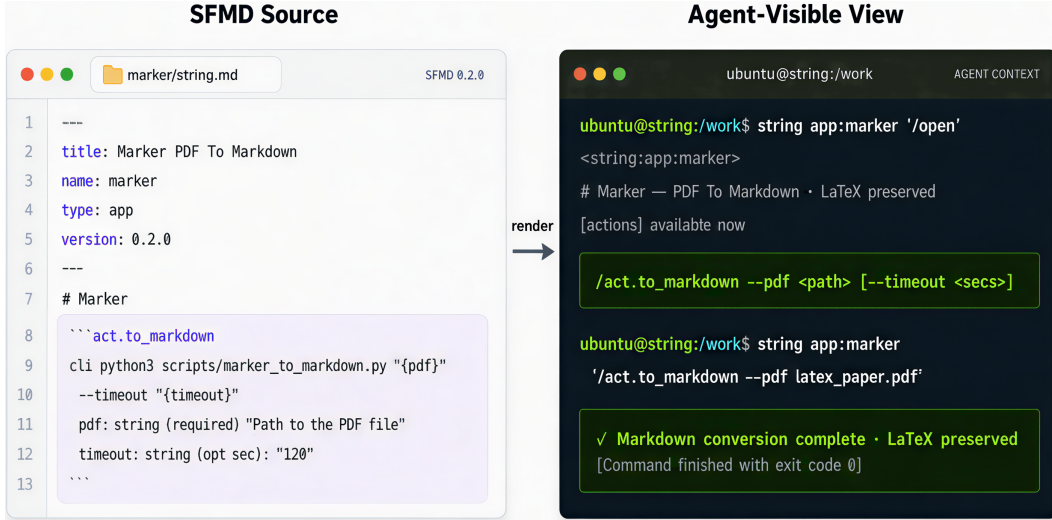


Figure 3: A String app and what the agent sees (schematic). Left: the SFMD source of marker (PDF→Markdown): frontmatter, page, one fenced act block wrapping a local script. Right: the agent-visible view—the page and an invocable signature; interpreter, script path, and argument plumbing never render (P1, P3).

How to read Table 3. The models differ in reasoning style and verbosity, so absolute token counts are not comparable across rows; the meaningful comparison is pre→post within a row. Success moves in whole tasks (each task is 5pp of the 20-task set): Qwen3.5-4B goes from 12/20 to 14/20 solved, Gemma 4-E4B from 13/20 to 16/20, Gemma 4-31B from 16/20 to 18/20, and Qwen3.6-27B holds at 17/20. Tokens per task fall by half to four-fifths in every row, including the row where success did not move—the fine-tuned models stop re-deriving the interface (exploratory opens, malformed commands, retries) and navigate directly to the acting command.

Table 3: Success and mean generated tokens per task before/after fine-tuning on String trajectories (20 held-out tasks).

| Model | Success (pre→post) | Generated tokens/task (pre→post) | Token cut |
|-------------|-----------------------|----------------------------------|-----------|
| Qwen3.5-4B | 12/20→14/20 (60%→70%) | 345.5→122.0 | 64.7% |
| Gemma 4-E4B | 13/20→16/20 (65%→80%) | 182.8→38.6 | 78.9% |
| Qwen3.6-27B | 17/20→17/20 (85%→85%) | 419.2→201.5 | 51.9% |
| Gemma 4-31B | 16/20→18/20 (80%→90%) | 200.2→43.7 | 78.2% |

Scope. This is an initial proof of concept, and we treat it as such: 20 tasks resolve success only to 5pp, and all four models were trained on trajectories from one deployment. What the study is evidence for is the co-design claim of §4—that a small, stable, runtime-absorbed interface is a *learnable* target, across two model families and two scales. The next round expands the trajectory data and the held-out set and adds standard benchmarks to test generalization beyond our own deployment.